Екстремно програмиране - 2

Никога не добавяйте функционалност по-рано

- Кодът за днес е за **днес**, а за утре оставете го до утре.
- **Избягвайте предположения** за това какво е необходимо в бъдеще.

Поддържайте системата незатрупана с допълнителни неща, които предполагате, че ще бъдат използвани по-късно.

Само 10% от тези допълнителни неща някога ще бъдат използвани, така че **губите 90% от времето си**.

Всички се изкушаваме да добавим функционалност сега, а не по-късно, защото виждаме как точно да я добавим или защото това би направило системата много по-добра.

Никога не добавяйте функционалност по-рано

Изглежда, че би било по-бързо да го добавите сега.

Но трябва постоянно да си напомняме, че всъщност няма да имаме нужда от това.

Допълнителната функционалност винаги ще ни **забавя** и ще **пропилява ресурсите**.

Затваряйте си очите за бъдещи изисквания и допълнителна гъвкавост. **Концентрирайте** се само върху това, което е насрочено за **днес**.

Когато бъде намерен бъг

- Създайте тестове, за да предотвратите **появата му отново**.
- Тестовете за приемане помагат на клиентите да **дефинират** и съобщят накратко проблема.
- Създаден тест **преди** отстраняване на грешки помага на разработчиците да:
  - съсредоточат усилията си (създайте **единични-** unit тестове).
  - знаят кога проблемът е отстранен.

Когато бъде намерен бъг

Когато се открие грешка, се създават тестове, за да се предпазят от допускане на същата грешка отново.

Грешка изисква да бъде написан тест за приемане, за да се предпази от нея.

Създаването на тест за приемане преди отстраняване на грешки помага на клиентите да дефинират кратко проблема и да го съобщят на програмистите.

Когато програмистите са създали неуспешен тест, трябва да съсредоточат усилията си и да узнаят дали проблемът е отстранен веднага.

Когато бъде намерен бъг

Като тестът за приемане е **неуспешен**, разработчиците могат да създадат **единични тестове**, за да покажат дефекта от гледна точка, специфична за изходния код.

Неуспешните модулни тестове дават незабавна **обратна връзка** за усилията за разработка, когато грешката е поправена.

Когато модулните тестове се изпълняват на 100%, тогава неуспешният тест за приемане може да бъде стартиран отново, за да се потвърди, че грешката е коригирана.

Планиране на итерации

- Това става в началото на всяка итерация за да:
  - Изберете потребителски истории (клиент) и неуспешни тестове за приемане при итерацията.
  - Превеждате избраните потребителски истории в задачи за програмиране.
  - Регистрирайте задачи и ги оцените (разработчици).
  - Използвайте скоростта на проекта и да определите дали задачите могат да бъдат изпълнени навреме.
- Планирайте само задачи за програмиране за първата (следващата) итерация.

Планиране на итерации

Среща за планиране на итерацията се свиква в началото на всяка итерация, за да се изготви **планът** на задачите за програмиране на тази итерация. Всяка итерация е с продължителност от 1 до 3 седмици. **Потребителските истории** се избират за тази итерация от клиента от плана по реда на най-ценното за клиента.

Избират се и **неуспешните** тестове за приемане, които трябва да бъдат коригирани. Потребителските истории и неуспешните тестове се превеждат в задачите за програмиране, които ще ги поддържат.

Задачите са записани на **карти**. Тези карти със задачи ще бъдат подробният **план за итерацията**.

Планиране на итерации

Всяка задача трябва да бъде с **продължителност от 1 до 3 идеални дни** за програмиране.

Идеалните дни за програмиране са колко време би ви отнело да изпълните задачата, ако нямаше разсейване. Задачите, които са по-кратки от 1 ден, могат да бъдат групирани. Задачите, които са по-дълги **от 3 дни**, трябва да бъдат разбити по-далеч. Разработчиците се регистрират за задачите и след това изчисляват колко време ще отнеме изпълнението на техните собствени задачи. Важно е разработчикът, който приема дадена задача, също да бъде този, който оценява колко време ще отнеме завършването. Скоростта на проекта се използва, за да се определи дали итерацията е над резервирана или не.

Планиране на итерации

Сумирайте прогнозното време в идеални дни за програмиране на задачите, това **не трябва да надвишава скоростта на проекта** от предишната итерация.

Ако итерацията има **твърде много време**, тогава клиентът трябва да избере потребителски истории, които да бъдат отложени за по-късна итерация (“снегоочистване”). Ако итерацията има **твърде малко време**, може да се приеме друга история. Скоростта в дните на задачите (планиране на итерации) отменя скоростта в седмиците на историята (планиране на пускане), тъй като е по-точна. Често е тревожно от историите на потребителите, които се изчистват от снега.

Не се паникьосвайте. Помнете **важността** на **юнит тестването и рефакторинга.**

Планиране на итерации

Избягвайте добавянето на каквато и да е функционалност, преди да е насрочено. Просто добавете каквото ви трябва за днес. Добавянето на нещо допълнително ще ви **забави**. Не се изкушавайте да промените прогнозите си за задача и история.

Процесът на планиране разчита на хладната реалност на последователните оценки, измислянето им да бъдат малко по-ниски създава повече проблеми.

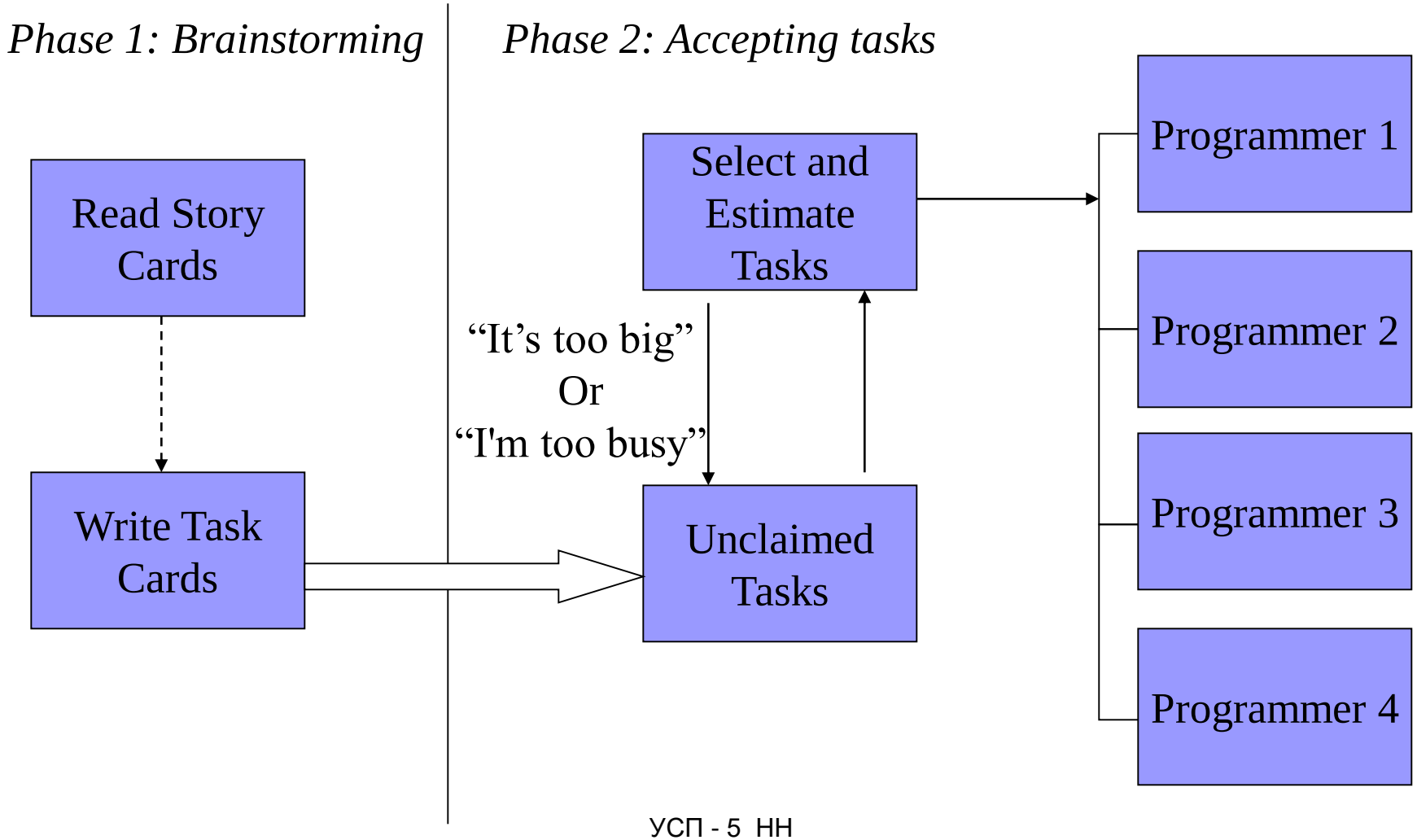
Следете скоростта на проекта и снегопочистването.

Планиране на итерации

Може да се наложи да **преоцените** всички истории и да договаряте плана за пускане на всеки три до пет повторения, това е нормално.

Докато винаги първо прилагате най-ценните истории, винаги ще правите възможно най-много за вашите клиенти и ръководство. Итеративният стил на разработка може да добави гъвкавост към вашия процес на разработка. Опитайте да планирате точно навреме, като не планирате конкретни задачи по програмиране по-далеч от текущата итерация.

Играта - планиране на итерации



Development

Zoom Out

Learn and Communicate

Unfinished Tasks

Pair Programming

Refactor Mercilessly

Move People Around

CRC Cards

Iteration Plan

Tasks

Too Much To Do

Share

New Functionality

Stand Up Meeting

Collective Code Ownership

100% Unit Tests Passed

Next Task or Failed Acceptance Test

Failed Acceptance Tests

Acceptance Test Passed

Day by Day

Bug Fixes

Copyright 2000 J. Donvan Wells

Ежедневни срещи

- Провеждата се всяка сутрин с цел комуникация за:
  - проблеми.
  - решения.
  - насърчава фокуса на екипа.
- Заменят срещите по проекта.

Ежедневни срещи

На типична **среща** по проекта повечето присъстващи не допринасят, а присъстват само за да чуят резултата. Голямо количество време на разработчиците се губи, за да се получи незначително количество комуникация. Наличието на много хора на всяка среща източва ресурси от проекта и също така създава кошмар за планиране.

Комуникацията между целия екип е целта на ежедневната среща. Среща в **изправено положение** всяка сутрин се използва за комуникация на проблеми, решения и насърчаване на фокуса на екипа. Всички се изправят в кръг, за да се **избегнат** дълги дискусии. По-ефективно е да имате една кратка среща, на която всеки трябва да присъства, отколкото много срещи с няколко разработчици.

Ежедневни срещи

Ежедневните срещи помагата да се редуцират хората при другите срещи до само необходимите членове на екипа.

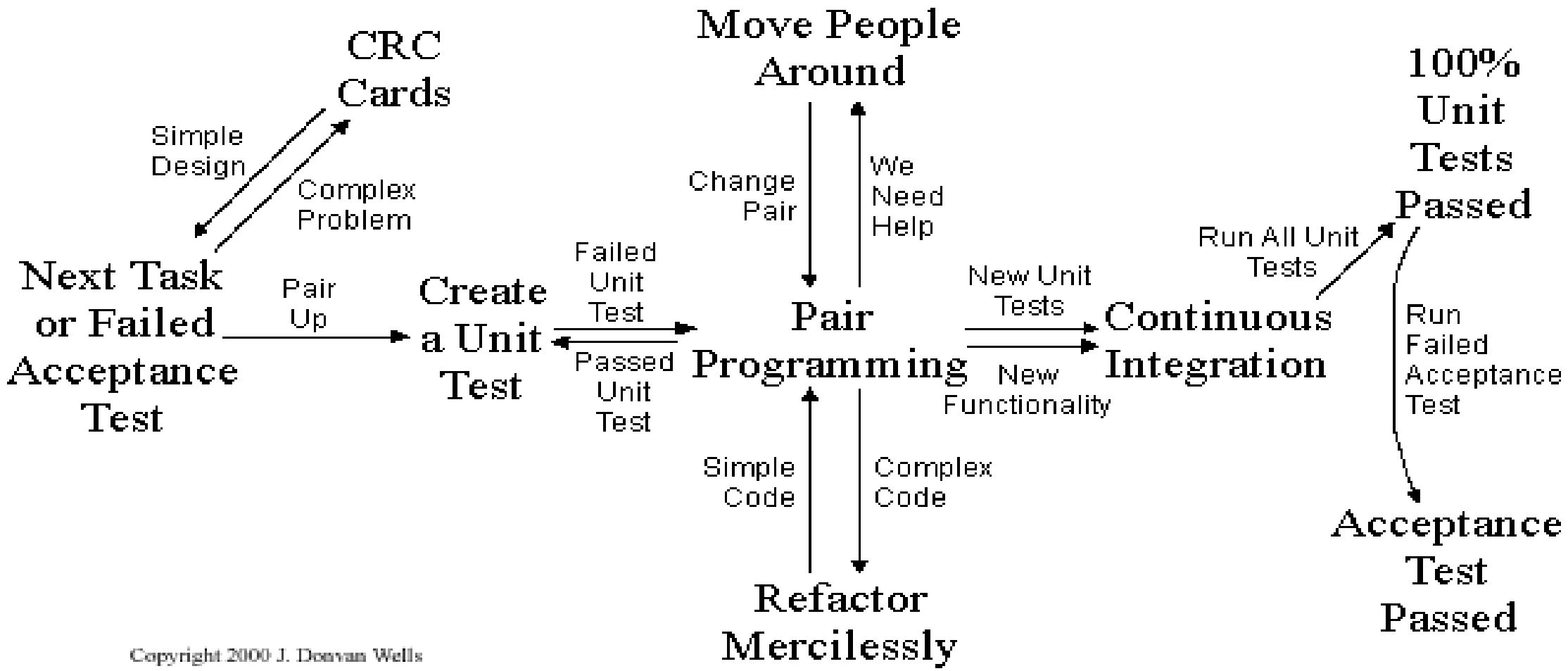
Така е възможно да се избегне дори планирането на повечето срещи.

При ограничено присъствие повечето срещи могат да се проведат спонтанно пред компютър, където кодът може да се преглежда и идеите действително изпробвани.

Ежедневната среща не е поредната среща за губене на времето на хората. Тя ще замени много други срещи, като ще осигури спестяване време от други по-продължитени срещи.

Collective Code Ownership

Zoom Out



Copyright 2000 J. Donvan Wells

I.Колективна собственост върху кода

Колективната собственост върху кода насърчава всеки да допринася с нови идеи за всички сегменти на проекта. Всеки разработчик може да промени всеки ред от код, за да добави функционалност, да коригира грешки или да преработи. Никой не се превръща в тясното място за промени. Това е трудно за разбиране в началото. Почти немислимо е цял екип да отговаря за архитектурата на системата. Да нямаш нито един главен архитект, който да поддържа жив някакъв визионерски пламък, изглежда, че не би могло да работи. Но не е необичайно да зададете въпрос на главен архитект и да получите отговор, който е просто грешен. Това не би бил провал на вашите водещи програмисти.

I.Колективна собственост върху кода

Всяка нетривиална система не може да се държи в съзнанието на един човек. Други програмисти работят усърдно, променяйки системата, без да се възползват от визията на архитекта. Независимо дали го осъзнавате или не, вашата архитектура **вече е разпределена между вашия екип**. Ако целият екип вече носи някаква отговорност за архитектурните решения, не трябва ли да получи и властта? Начинът, по който това работи, е всеки разработчик да създава единични тестове за своя код, докато се разработва. Целият код, който се пуска в хранилището на изходния код, включва модулни тестове. Добавяният код, коригираните грешки и старата функционалност при промяната ще бъдат обхванати от автоматизирано тестване.

I.Колективна собственост върху кода

Сега можете да разчитате на тестовия пакет, за да наблюдавате цялото ви хранилище с кодове. Преди всеки код да бъде пуснат, той трябва да премине целия тестов пакет на 100%. След като това е на място, всеки може да направи промяна на всеки метод от всеки клас и да го освободи в хранилището на код, ако е необходимо. Когато се комбинират с честа интеграция, разработчиците рядко дори забелязват, че даден клас е бил разширен или поправен. На практика колективната собственост върху код всъщност е **по-надеждна от поставянето на едно лице**, отговорно за гледането на конкретни класове. Още повече, че човек може да напусне проекта по всяко време.

II.Колективна собственост върху кода

- Избягва тесните места.
- Дава защита чрез модулните тестове (целият код трябва да включва тестове за модули).
- Позволява целия код да премине целия тестов пакет на 100%.

CRC карти

- Картите за клас, отговорности и сътрудничество (CRC) се използват за проектиране на системата като екип.
- Позволява на целия екип да участва.
- Симулира системата, като представя обектите с отговорностите и съвместните обекти.

CRC карти

Използвайте карти за **клас, отговорности и сътрудничество** (**CRC**), за да проектирате системата като екип. Най-голямата стойност на CRC картите е да позволят на хората да се откъснат от процедурния начин на мислене и да оценят по-пълно обектната технология.

CRC карти позволяват на цели екипи по проекти **да допринесат** за дизайна. Колкото повече хора могат да помогнат при проектирането на системата, толкова по-голям е броят на **добрите идеи**, включени. Индивидуалните CRC карти се използват **за представяне на обекти**. Класът на обекта може да бъде написан в горната част на картата, отговорностите са изброени отляво, класовете за сътрудничество са изброени вдясно от всяка отговорност.

CRC карти

Казваме „може да бъде написано“, защото след като сесията на CRC е в разгара си, участниците обикновено се нуждаят само от няколко карти с името на класа и на практика няма карти, изписани изцяло.

CRC сесията продължава с някой, който симулира системата, като говори за това кои обекти изпращат съобщения до други обекти.

Чрез преминаване през процеса слабостите и проблемите **лесно се разкриват**. Алтернативите на дизайна могат да бъдат проучени бързо чрез симулиране на предлагания дизайн. Ако откриете твърде много хора, които говорят и движат карти наведнъж, просто ограничете броя на хората, които стоят и местят карти до двама.

CRC карти

Когато един човек седне, друг може да се изправи.

Това работещ начин за сесии, които излизат извън контрол, което често се случва, когато екипите стават скандални, когато най-накрая се реши труден проблем.

Една от най-големите **критики** към CRC карти е липсата на писмен дизайн. Това обикновено не е необходимо, тъй като CRC карти правят дизайна да изглежда очевиден. Ако се изисква по-постоянен запис, една карта за всеки клас може да бъде изписана изцяло и запазена като документация. Един дизайн, веднъж замислен така, сякаш вече е построен и работи, остава с човек за известно време.

CRC карти

- Могат да се използват и **UML-инструменти**, но може да са неудобни, когато участват много хора.
- Възможен е **двупосочен инженеринг** в много „CASE“-инструменти за UML моделиране на CRC-дизайна от сорс кода.

Първо кодирайте модулните тестове

- По-лесно и по-бързо е при създаване на код, ако **модулните тестове** са създадени преди кода.
- Позволяват точно да се оцени обхвата на внедряването.
- Помагат да се прецени какво трябва да се направи. Изискванията са представени чрез тестове.
- Дава се незабавна **обратна връзка** и ние знаем кога сме готови (всички тестове се изпълняват).
- Нараства увереността в правилното функциониране на системата, докато расте.

Първо кодирайте модулните тестове

Ако първо създадете своите тестове, преди кода, ще откриете, че е много по-лесно и по-бързо да създадавате своя код. Комбинираното време, необходимо за създаване на модулен тест и създаване на код, който да го накара да премине, е приблизително същото като простото кодиране веднага. Но ако вече имате модулните тестове, не е необходимо да ги създавате след кода, което ви спестява известно време сега и много по-късно. Създаването на единичен тест помага на разработчика наистина да обмисли какво трябва да се направи. Изискванията се определят твърдо чрез тестове. Не може да има недоразумение спецификация, написана под формата на изпълним код.

Първо кодирайте модулните тестове

Освен това имате незабавна обратна връзка, докато работите. Често не е ясно кога разработчикът е завършил цялата необходима функционалност. Може да възникне пълзене на обхвата, когато се разглеждат разширения и условия за грешка. Ако първо създадем нашите единични тестове, тогава знаем кога сме готови; всички тестове на модула се изпълняват.

Има и полза от системния дизайн. Често е много трудно да се създане единичният тест на някои софтуерни системи. Тези системи обикновено са първо изградени с код и второ тестване, често изцяло от различен екип. Като създадете първо тестове, вашият дизайн ще бъде повлиян от желанието да тествате всичко ценно за вашия клиент.

Първо кодирайте модулните тестове

Вашият дизайн ще отрази това, като бъде по-лесен за тестване. Първо има **ритъм за разработване** на софтуерен модулен тест. Създавате един тест, за да дефинирате някакъв малък аспект на проблема. След това създавате най-простия код, който ще направи този тест да премине. След това създавате втори тест. Сега добавяте към кода, който току-що създадохте, за да преминете този нов тест, но не повече! Не и докато имате още трети тест. Продължавате, докато не остане нищо за тестване.

Кодът, който ще създадете, **е прост и компактен**, като прилага само функциите, които желаете. Други разработчици могат да видят как да използват този нов код, като преглеждат тестовете. Входът, чиито резултати са недефинирани, ще отсъства видимо от тестовия пакет.

Първо кодирайте модулните тестове

- Полза от предварителното проектиране на системата, е че става по-лесна за тестване.
- Разделя логическия дизайн от физическото изпълнение.
- Опростява дизайна.
- Винаги кодирайте възможно най-просто, за да преминете тестовете.
- Кодирани са само желаните функции.

I: Програмиране по двойки

- Целият продуциран код се създава от **двама души**, работещи заедно на един компютър.
- Повишава се **качеството** на софтуера без да се влияние върху времето за доставка.
- Добавяте колкото се може повече функционалност, като кодирате отделно с много по-високо качество (спестява време по-късно в проекта).

I: Програмиране по двойки

Целият код, който трябва да бъде включен в производствената версия, се създава от двама души, работещи заедно на един компютър. Програмирането по двойки повишава качеството на софтуера, без да се отразява на времето за доставка. Това е противоположно интуитивно, но 2-ма души, работещи на един компютър, ще добавят толкова функционалност, колкото двама, работещи поотделно, с изключение на това, че качеството ще бъде много по-високо.

С повишеното **качество** идват големи спестявания по-късно за проекта

I: Програмиране по двойки

Най-добрият начин за сдвояване на програмата е просто да седнете един до друг пред монитора.

Плъзнете клавиатурата и мишката напред-назад. Един човек пише и мисли тактически за метода, който се създава, докато друг мисли стратегически за това как този метод се вписва в класа.

Отнема време, за да свикнете с програмирането по двойки, така че не се притеснявайте, ако в началото се почувствате неудобно.

II: Програмиране по двойки

- Единият програмист мисли тактически за метода за прилагане.
- Другият мисли стратегически за това как този метод се вписва в класа.
- Повече хора научават повече за системата.
- Двойките не са постоянни, могат да се сменят много пъти на ден.

Ключът е в простотата

- Простият дизайн отнема по-кратко време за завършване от сложния.
- Винаги правете най-простото нещо, което може да работи.
- Заменете сложния код с по-опростен, по-евтино е да го замените сега, отколкото да използвате много време за него по-късно.
- Никога не добавяйте функционалност, преди да е планирана.
- Поддържането на прост дизайн е трудна работа!

Ключът е в простотата

Един **прост дизайн** винаги отнема по-малко време за завършване от сложния. Така че винаги правете най-простото нещо, което може да работи. Ако намерите нещо сложно, заменете го с нещо просто. Винаги е по-бързо и по-евтино да замените сложен код сега, преди да загубите много повече време за него. Поддържайте нещата възможно най-прости колкото е възможно по-дълго, като никога не добавяте функционалност, преди да е планирана от заявителя.

Внимавайте обаче, поддържането на прост дизайн е **тежка работа**.

Премествайте хората

- Така се **избягва** сериозна загуба на знания и проблеми с кодирането.
- Постига се **кръстосано обучение**.
- Всеки знае голяма част от кода за всяка част на системата.
- Прави екипа **по-гъвкав** и повишава **производителността**.
- Насърчавайте хората да работят върху нова част през всяка итерация.

Премествайте хората

Премествайте хората наоколо, за да избегнете сериозна **загуба** на **знания** и проблеми с кодирането. Ако само един човек от вашия екип може да работи в дадена област и този човек напусне или просто имате много неща, които чакат да бъдат свършени в този раздел, ще откриете, че напредъкът на вашия проект е намален до обхождане**.**

Кръстосаното обучение често е важен фактор в компаниите, които се опитват да избегнат острови на знания, които са толкова податливи на загуба. Преместването на хора около кодовата база в комбинация с програмиране по двойки прави кръстосано обучение вместо вас. Вместо един човек, който знае всичко за дадена част от кода, всеки от екипа знае голяма част от кода във всеки раздел.

Премествайте хората

Екипът е много **по-гъвкав,** ако всеки знае достатъчно за всяка част от системата, за да работи върху нея. Вместо няколко души да са претоварени с работа, докато другите членове на екипа нямат много работа, целият екип може да бъде продуктивен. Произволен брой разработчици могат да бъдат назначени към най-горещата част на системата. Гъвкавото балансиране на натоварването от този тип е сбъдната мечта на мениджъра. Просто насърчете всички да опитат да работят върху нов раздел от системата поне част от всяка итерация. Програмирането по двойки прави възможно без загуба на производителност и гарантира непрекъснатост на мисълта. Един човек от двойката може да бъде разменен, докато другият продължава с нов партньор, ако желаете.

Безмилостно преработвайте

- **Премахвайте** излишъка.
- Всичко трябва да бъде изразено веднъж и **само веднъж**.
- **Елиминирайте** неизползваната функционалност.
- **Заменяйте** сложен и/или остарял дизайн с нов по-опростен дизайн.
- Так се **спестява време** и се повишава **качеството**.
- Поддържайте кода **чист** и **кратък**.

Безмилостно преработвайте

Ние, компютърните програмисти, държим на нашите софтуерни проекти дълго след като са станали тромави. Продължаваме да използваме и използваме повторно код, който вече не може да се поддържа, защото все още работи по някакъв начин и се страхуваме да го модифицираме. Но наистина ли е рентабилно това? Екстремното програмиране (XP) заема позицията, че **не е така**. Когато премахваме излишъка, премахваме неизползваната функционалност и подмладяваме остарели дизайни, ние преработваме. **Рефакторингът** (преработката) през целия жизнен цикъл на проекта спестява време и повишава качеството. Преработвайте безмилостно, за да поддържате дизайна прост, докато отивате и да избягвате ненужното претрупване и сложност.

Безмилостно преработвайте

Поддържайте **кода си чист и сбит**, за да е по-лесен за разбиране, модифициране и разширяване. Уверете се, че всичко е изразено веднъж и само веднъж. В крайна сметка отнема по-малко време за създаване на система, която е добре поддържана.

Има известно количество мистика в рефакторинга. В началото е трудно, защото трябва да можете да се откажете от този перфектен дизайн, който сте си представили, и да приемете дизайна, който е бил случайно открит за вас чрез рефакторинг.

Безмилостно преработвайте

Трябва да осъзнаете, че дизайнът, който сте си представяли, е бил добър пътеводител, но вече е **остарял**.

Една гъсеница е идеално проектирана да яде огромни количества зеленина, но не може да намери половинка, трябва да се преработи в пеперуда, преди да бъде проектирана да търси в небето други от своя вид.

Освободете се от представите си за това каква трябва или не трябва да бъде системата и се опитайте да видите новия дизайн, какъвто се появява пред вас.

I: Интегрирайте често

- Интегрирайте и пускайте код в хранилището на кода възможно **най-често**.
- Никога **не задържайте** смяна повече от един ден.
- **Избягва** разминаващи се и фрагментирани усилия за развитие.
- Това прави **повторното използване** и споделянето на код по-ефективни.

I: Интегрирайте често

Разработчиците трябва да интегрират и пускат код в кодовото хранилище на всеки няколко часа, когато е възможно. Във всеки случай никога не задържайте промените **повече от един ден**. Непрекъснатата интеграция често избягва различни или фрагментирани усилия за разработка, при които разработчиците не комуникират помежду си за това какво може да се използва повторно или какво може да бъде споделено.

Всеки трябва да работи с **най-новата версия**. Не трябва да се правят промени в остарелия код, причиняващи главоболия при интеграцията.

I: Интегрирайте често

Почти непрекъснатата интеграция **избягва** или **открива проблеми** със съвместимостта рано. Интеграцията е вид дейност „платете ми сега или ми платете повече по-късно“.

Тоест, ако интегрирате проекта в малки количества, няма да откриете, че се опитвате да интегрирате системата седмици наред в края на проекта, докато крайният срок изтече.

Винаги работете в контекста на най-новата версия на системата.

II: Интегрирайте често

- Всички работят с най-новата версия.
- Проблемите със съвместимостта са открити и/или избегнати по-рано.
- Избягва проблеми с интеграцията в края на проекта.

Последователна интеграция изисква:

- Разработчиците трябва да интегрират кода в хранилището за кодове последователно. Само една двойка веднъж.
  - Последователно хранилище.
  - Избягвайте остарелите версии.
- Да има специализиран компютър за целта.
- Интегрирайте и пускайте често.

Последователна интеграция

Без да контролират интеграцията на изходния код, разработчиците тестват своя код и интегрират, вярвайки, че всичко е наред. Но поради паралелно интегриране на модули с изходен код има комбинация от изходен код, която не е тествана заедно преди. Множество проблеми възникват с интеграцията без да бъдат откривани. Допълнителни проблеми възникват, когато няма ясна най-новата версия. Това се отнася не само за изходния код, но и за комплекта за тестове на модули, който трябва да провери коректността на изходния код. Ако нямата пълен, правилен и последователен **тестов пакет**, вие ще преследвате грешки, които не съществуват, и ще пропускате грешки, които съществуват.

Последователна интеграция

Някои проекти се опитват да имат разработчиците да притежават специфични класове. След това собствениците на класа гарантират, че кодът за всеки клас е интегриран и пуснат правилно. Това намалява проблема, но междукласовите зависимости все още могат да бъдат грешни. Това не решава целия проблем. Друг начин е да назначите **интегратор** или интеграционен екип. Интегрирането на код от множество разработчици е повече, отколкото един човек може да се справи. А екип от хора е твърде голям ресурс, за да се интегрира повече от веднъж седмично. В тази среда разработчиците работят с остарели версии, които след това погрешно се интегрират отново в кодовата база. Тези решения не се адресират към основния проблем.

Последователна интеграция

Искате разработчиците да могат да продължат паралелно, като смело правят промени във всяка част от системата, която се изисква, но също така искате интегриране на тези усилия без грешки. Подобно на дузина парни локомотива, които се насочват към стрелочната станция едновременно, ще има проблеми. Вместо да ограничаваме развитието до последователност или да изискваме сложни процедури за интеграция, нека преосмислим проблема. Всички наши локомотиви могат да влязат в комбината без катастрофа, ако просто се **редуват**. Трябва да направим това и с интегрирането на кода. Строго последователната (или еднонишкова) интеграция от самите разработчици в комбинация с колективна собственост на код е просто решение на този проблем.

Последователна интеграция

Целият изходен код се освобождава в безопасно изходния код или в хранилище, като се редува. Тоест само една двойка за разработка интегрира, тества и пуска промени в хранилището на изходния код във всеки даден момент.

Еднонишковата интеграция позволява последователно да се идентифицира най-новата версия. Това не означава, че не можете да интегрирате собствените си промени с най-новата версия на собствената си работна станция по всяко време, когато пожелаете.

Последователна интеграция

Просто не можете да пуснете промените си в екипа, без да чакате вашия ред. Изисква се някакъв механизъм за заключване. Най-простото нещо е физически токен, предаван от разработчик на разработчик.

Един компютър, предназначен за тази цел, работи добре, ако екипът за разработка е съвместно разположен. Интегрирането и освобождаването на код често съкращава времето, необходимо за задържане на ключалката и по този начин намалява времето за изчакване за придобиване на ключалката.

I: Тестване на модулите

- Създайте/изтеглете фреймуорка за единични тестове (т.е. JUnit).
- Създайте автоматизирани тестови пакети.
- Тествайте всички класове в системата.
- Създайте тестове преди кода.
- Единични тестове са част от изданието.
- Код без тест не трябва да се пуска.

I: Тестване на модулите

Единичните тестове са един от крайъгълните камъни на Extreme Programming (XP). Но стилът на единичните тестове XP е малко по-различен.

Първо трябва да създадете или изтеглите фреймуорк за модулни тестове, за да можете да създавате автоматизирани пакети за тестове за модули.

Второ, трябва да тествате всички класове в системата. Тривиалните методи за получаване и настройка обикновено се пропускат. И последно, първо трябва да опитате да създадете своите тестове, преди кода. Единичните тестове се пускат в кодовото хранилище заедно с кода, който тестват. Код без тестове може да не бъде пуснат.

I: Тестване на модулите

Ако се установи, че липсва единичен тест, той трябва да бъде създаден по това време. Най-голямата съпротива срещу отделянето на този период от време за единични тестове е бързо наближаващият краен срок. Но по време на живота на даден проект автоматизираният тест може да ви спести сто пъти повече от разходите за създаването му, като намира и предпазва от грешки.

Колкото по-труден е тестът да напишете, толкова повече имате нужда от него, защото толкова по-големи ще бъдат спестяванията ви. Автоматизираните единични тестове предлагат възвръщаемост, много по-голяма от цената на създаването. Друго често срещано погрешно схващане е, че единичните тестове могат да бъдат написани през последните три месеца от проекта.

I: Тестване на модулите

За съжаление, без модулните тестове развитието се проточва и изяжда и след последните три месеца. Дори ако времето е налично, на добрите пакети за единични тестове е необходимо време, за да се развият. Откриването на всички проблеми, които могат да възникнат, отнема време. За да имате пълен набор от тестове за единици, когато имате нужда от него, трябва да започнете да създавате тестовете веднага. Единичните тестове позволяват колективно притежаване на код. Когато създавате модулни тестове, вие предпазвате своята функционалност от случайно увреждане. Изискването на целия код да премине всички модулни тестове, преди да може да бъде пуснат, гарантира, че цялата функционалност винаги работи. Притежанието на код не се изисква, ако всички класове са защитени от тестове за модули.

II: Тестване на модулите

- Трябва да се създадат липсващи единични тестове.
- Позволява се колективна собственост върху код.
- Предпазва функционалността от случайна повреда.
- Целият код трябва да премине всички модулни тестове, преди да бъде пуснат.

Поправете XP, когато се счупи

Ако има практики, които не работят, **променете ги**.

Поправете процеса, когато се счупи. Не казваме дали, тъй като вече знаем, че ще трябва да направите някои промени за вашия конкретен проект. Следвайте правилата на XP, за да започнете, но не се колебайте да промените това, което не работи. Това не означава, че екипът може да прави каквото си поиска. Правилата трябва да се спазват, докато екипът не ги **промени**. Всички ваши разработчици трябва да знаят точно какво да очакват един от друг, наличието на набор от правила е единственият начин да зададете тези очаквания.

Правете **срещи**, за да обсъдите какво работи и какво не и да измислите начини за подобряване на XP.

Клиентът е винаги на разположение

- Едно от малкото изисквания.
- **Той е част от екипа**. Всички фази изискват комуникация с клиента като:
  - Потребителски истории.
  - Планиране на освобождаването.
  - Тестове за приемане.
  - Подробни потребителски истории.
  - Ранно изпробване на системата.

Клиентът е винаги на разположение

Едно от малкото изисквания на екстремното програмиране (XP) е клиентът да е **на разположение**. Не само да помогне на екипа за разработка, но и да бъде част от него. Всички фази на XP проект изискват **комуникация** с клиента, за предпочитане лице в лице, на място.

Най-добре е просто да **назначите** един или повече клиенти към екипа за разработка. Внимавайте обаче, това да не за дълго време. Вие имате нужда от него като експерт.

Потребителските истории се пишат от клиента, като това помага на разработчиците, за да правилни оценки на времето и зададените приоритети.

Клиентът е винаги на разположение

Използването на клиентите помага да се **гарантира**, че повечето от желаната функционалност на системата е обхваща потребителските истории. По време на срещата за планиране на пускането на версията клиентът ще трябва да договори избор на потребителски истории, които да бъдат включени в нея. Може да се наложи да се договори и **времето** за пускане на весриите.

Клиентите трябва да вземат решенията, които засягат техните **бизнес цели**. Среща за планиране на изданието се използва за дефиниране на малки постепенни издания, за да се позволи функционалността да бъде пусната на клиента по-рано. Това дава възможост на клиентите да изпробват системата по-рано и да дадат обратна връзка на разработчиците по-рано.

Организация на проекта XP- екип

- Customer - Клиент
- Programmers - Програмисти
- Project Manager -(Ръководител на проекта)
- Tracker - Проследител
- Coach -Треньор

На XP мениджърите се препоръчва:

- Не определяйте приоритети:- клиентът го прави
- Не поставяйте задачи: -програмистите го правят
- Не преценявайте истории или задачите: програмистите го правят
- Не диктувайте графици: клиентът и програмистите ги договарят

XP мениджърите организира:

- Външни събирания.
- Формирането на екипа.
- Получаването на ресурси.
- Управляването на екипа:
- Докладването за напредъка.
- Домакинстването на срещи.
- Домакинстването на събрания.
- Управляването на проблемите.

Tracker – Проследителят се ангажира с:

- Проследяване на плана за пускане (истории)
- Проследяване на итерационния план (задачи)
- Проследяване на тестове за приемане (успешни, неуспешни)

I: Как да се стартира XP?

- При нов проект:
  - Събирайте потребителските истории.
  - Реализирайте спайк решения.
  - Срещайте се за планиране на реализацията.
  - Среща за итеративно развитие (среща за планиране на итерации).

I: Как да се стартира XP?

Най-очевидният начин да започнете екстремно програмиране (XP) е с **нов проект**. Започнете да **събирате потребителски истории** и да прилагате решения за върхове за неща, които изглеждат рискови.

Прекарайте само няколко седмици, правейки това. След това насрочете среща за **планиране на реализацията**.

Поканете клиенти, разработчици и мениджъри да създадат график, с който всички са съгласни. Започнете своето **итеративно развитие** със среща за планиране на итерации. Сега сте започнали. Обикновено проектите запчоват да търсят нова методология като XP само след като проектът е в беда и не се развива.

I: Как да се стартира XP?

В този случай най-добрият начин да стартирате XP е да разгледате добре **текущата си софтуерна методология** и да разберете какво ви забавя. Първо добавете XP към този проблем.

Например, ако установите, че 25% от пътя на вашия процес на разработка, спецификацията на вашите изисквания става напълно безполезна, тогава се съберете с клиентите си и запишете потребителските истории. Ако имате хроничен проблем с промяната на изискванията, което ви кара често да преразглеждате графика, опитайте с по-ясна **среща за планиране** на издаване на версията на всеки няколко цикъла. (Първо обаче ще ви трябват потребителски истории.)

I: Как да се стартира XP?

Опитайте **итеративен стил** на разработка и точен стил на планиране на задачите по програмиране.

Ако е проблем броя на грешките в производството, опитайте с **автоматизирани функционални тестове**. Използвайте този набор от тестове за тестване чрез регресия и валидиране.

Ако са проблем грешки в интеграцията, опитайте с автоматизирани тестове за **модулите**.

Изисквайте всички модулни тестове да преминат (100%), преди всеки нов код да бъде пуснат в хранилището на кодове.

II: Как да се стартира XP?

- При случай, че проектът вече се изпълнява.
  - Събирайте **потребителски истории**, ако имате проблеми с изискванията.
  - Итеративно и точно навреме планирайте задачите при промяна на изискванията.
  - Използвайте автоматизирани функционални тестове, ако има много грешки в производството, които да се използват при тестване на регресия и валидиране.
  - Прилагайте автоматизирани модулните тестове, ако има проблеми с интеграцията.
  - Използвайте рефакторинг и тестово програмиране (самостоятелно).

Правилата и практики на екстремното програмиране.

- **Планиране**.
  - Пишете потребителските истории.
  - Планирайте реализацията и създайте график.
  - Правете чести малки издания.
  - Измервайте скоростта на проекта.
  - Разделяйте проектъта на итерации.
  - Планиранейте началото на всяка итерация.
  - Премествайте хората наоколо.
  - Всеки ден започваите с кратка среща.
  - Поправете XP, когато се счупи.

Правилата и практики на екстремното програмиране.

- **Дизайн**.
  - Използвайте простота.
  - Изберете системна метафора.
  - Използвайте CRC карти за дизайнерски сесии.
  - Създайте решения за скокове за намаляване на риска.
  - Никаква функционалност да не се добавя по-рано.
  - Рефакторирайте винаги, когато и където е възможно.

Благодаря за вниманието!

УСП - 3 НН